



Data Visualization

LABORATORIO

Capitolo 10 – Interazione



Argomenti del giorno

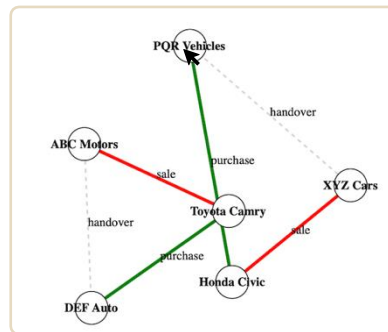
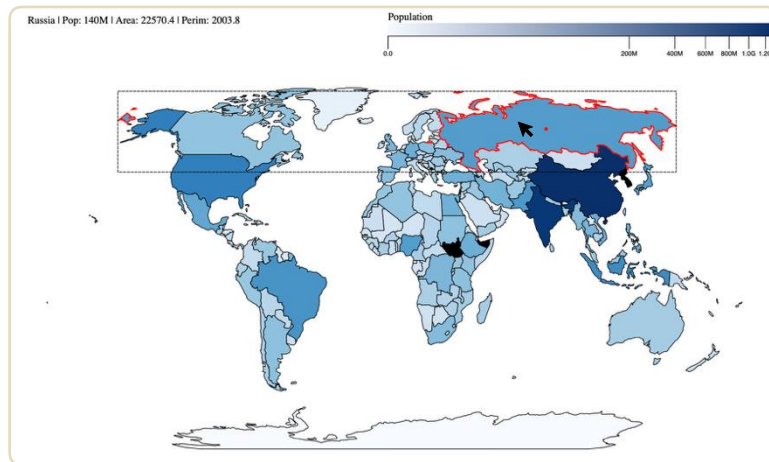
- **Filtraggio delle informazioni**
- **scaleOrdinal**
 - Schema colori personalizzato
 - Simboli
- **Brush**
 - Monodimensionali
 - Bidimensionali

Interazione tra grafici

Finora abbiamo visto come l'utente può interagire con un grafico:

- **Hovering** del mouse: l'elemento viene evidenziato.
- **Drag & Drop**: l'elemento viene spostato ma mantiene una logica.
- **Click** del mouse: l'elemento viene selezionato ed eventualmente modificato

Non abbiamo mai visto però come due grafici nella stessa pagina possono **interagire tra loro**

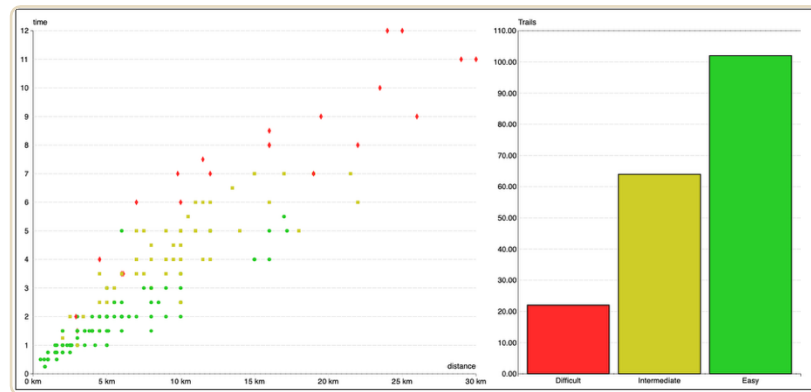


Filtri interattivi

Dato un dataset relativo a percorsi di trekking, il nostro obiettivo è far comunicare:

- Uno **scatterplot** che mostra la relazione tra distanza e tempo di percorrenza di percorsi.
- Un **barchart** che mostra la distribuzione dei percorsi per livello di difficoltà.

Quando **clicchiamo** su una barra dell'istogramma, i punti corrispondenti a quella difficoltà nello scatterplot verranno filtrati, **mostrando** o **nascondendo** solo quelli selezionati.



scaleColor personalizzata

La prima cosa da fare è andare a definire una **scaleColor** per i nostri grafici: finora abbiamo utilizzato uno schema colori di default di d3, ma se volessimo crearne una da zero?

La logica è molto semplice: ogni elemento del vettore del dominio va **mappato** in un colore del codominio.

Usiamo dei colori rappresentativi per le difficoltà e non colori casuali.

```
renderScatterplotBarchart()
```

```
// Stato condiviso del filtro sulle difficoltà e palette colori dedicata.  
let difficultyFilter = ["Easy", "Intermediate", "Difficult"];
```

```
// Creazione scale color custom per le difficoltà  
const scaleColorDifficulty = d3.scaleOrdinal()  
  .domain(difficultyFilter)  
  .range(["#29CD29", "#CDCD29", "#FF2929"]);
```

```
scatterplot(data, config.leftChart, svg, ['distance', 'time'], scaleColorDifficulty);  
barchart(data, config.rightChart, svg, 'ascending', 'difficulty', scaleColorDifficulty);
```

Simboli per classi

Per **diversificare** gli elementi, oltre che ai colori, possiamo utilizzare diversi **simboli**. D3 mette a disposizione una libreria di simboli, che noi possiamo usare per mappare le difficoltà con una scale function.

barchart ()

```
// barre del barchart, colorate in base alla difficoltà tramite la scala scaleColorDifficulty.
gContainer.append('g').classed("rects", true).selectAll('rect')
    .data(agggregatedDataMap)
    .join('rect')
    .attr('x', get_x)
    .attr('y', get_y)
    .attr('width', get_width)
    .attr('height', get_height)
    .attr('fill', d => scaleColorDifficulty(d[0]))
    .attr('stroke', 'black')
    .append('title')
    .text(d => d3.format(".2s")(d[1]))
```

scatterplot ()

Se vogliamo i simboli

```
// Mostrare il contenuto dello scatterplot con o senza simboli in base al flag useSymbols
// https://d3js.org/d3-shape/symbol
// Con 'useSymbols' decido se usare forme diverse per difficulty o semplici cerchi.
const useSymbols = false;
if (useSymbols) {
    // Se uso simboli, definisco una scala apposita per associare una forma a ogni categoria di difficoltà.
    const scaleSymbols = d3.scaleOrdinal()
        .domain(["Easy", "Intermediate", "Difficult"])
        .range([
            d3.symbol().type(d3.symbolCircle),
            d3.symbol().type(d3.symbolSquare),
            d3.symbol().type(d3.symbolDiamond)
        ])
    // Creo i simboli con la forma e il colore corrispondente alla difficoltà.
    gContainer.append('g')
        .classed('circles', true)
        .selectAll(null)
        .data(data)
        .join('path')
        .attr('class', 'symbol')
        .attr('transform',
            d => `translate(${scaleX(d[xKey])}, ${scaleY(d[yKey])})`)
        .attr('d', d => scaleSymbols(d.difficulty)())
        .attr("fill", d => scaleColorDifficulty(d.difficulty))
}
```

Se non vogliamo i simboli

```
} else {
    // Se non uso simboli, mostro semplici cerchi colorati in base alla difficoltà.
    gContainer.append("g")
        .selectAll(null)
        .data(data)
        .join("circle")
        .attr("cx", d => scaleX(d[xKey]))
        .attr("cy", d => scaleY(d[yKey]))
        .attr("r", 5)
        .attr("fill", d => scaleColorDifficulty(d.difficulty))
        .attr('stroke', 'black')
}
```

Raggruppare per classi

Per creare il bar chart, dobbiamo contare i percorsi e dividerli per difficoltà, le quali agiranno da classi nel nostro grafico.

La funzione **d3.rollups()** permette, dato un dataset, di ottenere il **numero di istanze** di una determinata **chiave**.

Il primo parametro preso in input è il dataset, il secondo è la **funzione di aggregazione**. In questo caso vogliamo la lunghezza, quindi il numero di istanze. Il terzo è la **chiave** per cui raggruppare, nel nostro caso "difficulty".

```
trail,region,difficulty,time,distance,season
Abby Grind,Fraser Valley East,Intermediate,1.5,4,year-round
Admiralty Point,Tri Cities,Easy,1.5,5,year-round
Al's Habrich Ridge Trail,Howe Sound,Intermediate,5,7,July - October
Aldergrove Regional Park,Surrey and Langley,Easy,2,5,year-round
Alexandra Bridge,Fraser Valley East,Easy,0.5,1.6,March - November
Alice Lake,Howe Sound,Easy,2,6,April - November
Ancient Cedars Trail,Whistler,Intermediate,2,5,June - October
Ascent Trail,Whistler,Difficult,3.5,6.1,July - September
```

barchart ()

```
const aggregatedDataMap =
  // Raggruppo i trail per difficulty e conto quante occorrenze ci sono per ogni gruppo.
  // Il risultato è un array di coppie
  d3.rollups(data, g => g.length, d => d[key]);

// Ordino le barre in base al numero di trail, in ordine crescente o decrescente a seconda del valore di sortBars.
if (sortBars === 'ascending')
  aggregatedDataMap
    .sort((a, b) => d3.ascending(a[1], b[1]));
else if (sortBars === 'descending')
  aggregatedDataMap
    .sort((a, b) => d3.descending(a[1], b[1]));
```

Re-rendering del grafico filtrato

Ora che i grafici sono pronti, creiamo la funzione che gestirà l'evento del **click** sulle barre.

Verifichiamo che la **classe** (difficoltà) del **bin** che clicchiamo sia inclusa nell'array di difficoltà:
se è presente la **rimuoviamo**,
se non c'è la **aggiungiamo**.

```
function updateRenderScatterplot(data, config, svg, keys, scaleColorDifficulty){  
  d3.select('svg > g.scatterplot').remove()  
  scatterplot(data, config, svg, keys, scaleColorDifficulty);  
}
```

renderScatterplotBarchart()

```
// Funzione di gestione del click sulle barre del barchart.  
const onClickDifficulty = function (event, d){ Show usages  
  // Si verifica se la difficoltà associata alla barra cliccata è già presente nel filtro.  
  const isBarFiltered = difficultyFilter.includes(d[0]);  
  // Se è già presente, significa che vogliamo rimuoverla dal filtro (escluderla dallo scatterplot),  
  // altrimenti la aggiungiamo.  
  if (isBarFiltered) {  
    difficultyFilter = difficultyFilter.filter(el => el !== d[0]);  
  }  
  else {  
    difficultyFilter.push(d[0]);  
  }  
  // Aggiorno lo stile della barra cliccata per riflettere se è attiva o meno nel filtro.  
  // Se la barra è attiva (inclusa nel filtro),  
  // ha un bordo nero e piena opacità, altrimenti è più trasparente e senza bordo.  
  d3.select(this)  
    .classed('clicked', difficultyFilter.includes(d[0]))  
    .attr('stroke-width', (d) => difficultyFilter.includes(d[0])? '2px': '0px')  
    .attr('stroke', (d) => difficultyFilter.includes(d[0])? 'black' : 'none')  
    .attr('opacity', (d) => difficultyFilter.includes(d[0])? '1' : '0.7')  
  
  // Dopo aver aggiornato il filtro, si filtra il dataset originale  
  // per includere solo i percorsi che corrispondono alle difficoltà attualmente selezionate.  
  const filteredData = data.filter(d => difficultyFilter.includes(d.difficulty));  
  // Infine, si ridisegna lo scatterplot con i dati filtrati, mostrando solo i percorsi delle difficoltà selezionate.  
  updateRenderScatterplot(filteredData, config.leftChart, svg, keys = ['distance', 'time'], scaleColorDifficulty);  
}  
  
// Associa la funzione di click a tutte le barre del barchart.  
d3.selectAll('.barchart .rects rect')  
  .on('click', onClickDifficulty)
```


Gestione radio-button

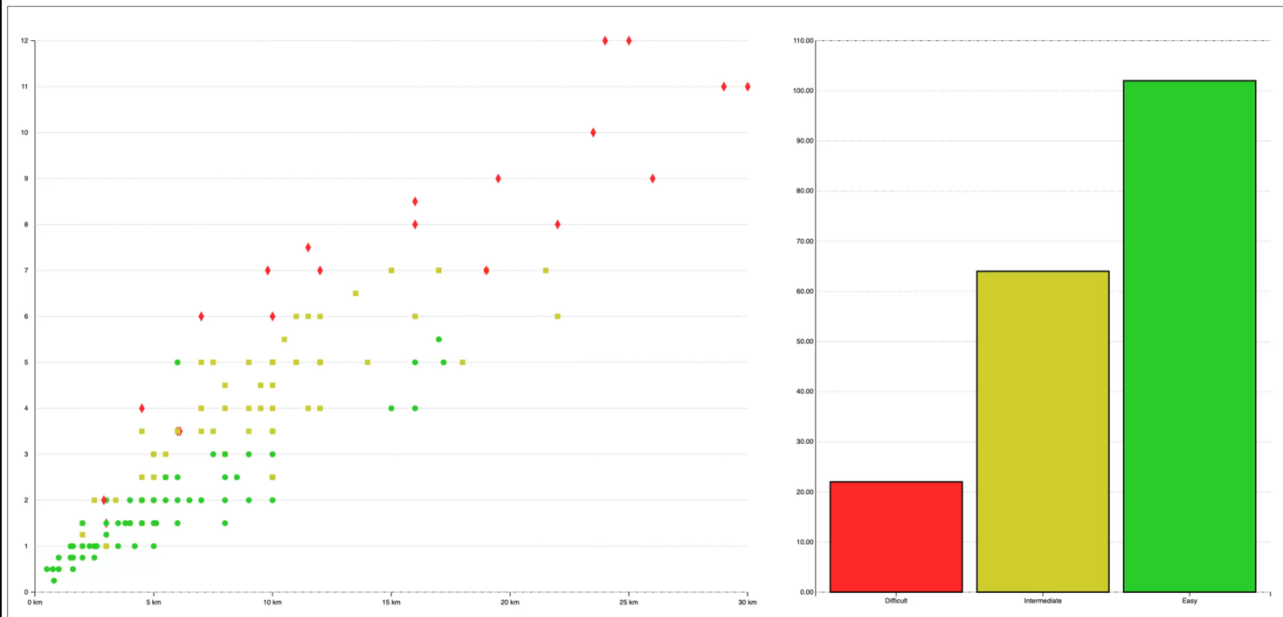
Adesso gestiamo il radio button per l'ordinamento delle barre. Aggiungiamo un event listener quando cambia il valore del radio button, di conseguenza, modifichiamo la disposizione delle barre

renderScatterplotBarchart()

```
// Gestisco il cambio di ordinamento tramite i radio button.  
d3.selectAll('input[name="sortOption"]').on('change', function(){  
  // Il cambio radio ricrea la barchart con il nuovo ordinamento.  
  let currentOrder = this.value;  
  d3.select('.barchart').remove();  
  barchart(data, config.rightChart, svg, currentOrder, 'difficulty', scaleColorDifficulty);  
  d3.selectAll('.barchart .rects rect')  
    .on('click', onClickDifficulty)  
});
```

Risultato

Scatterplot e BarChart



Ordinamento

- ☒ Crescente
- ☐ Decrescente

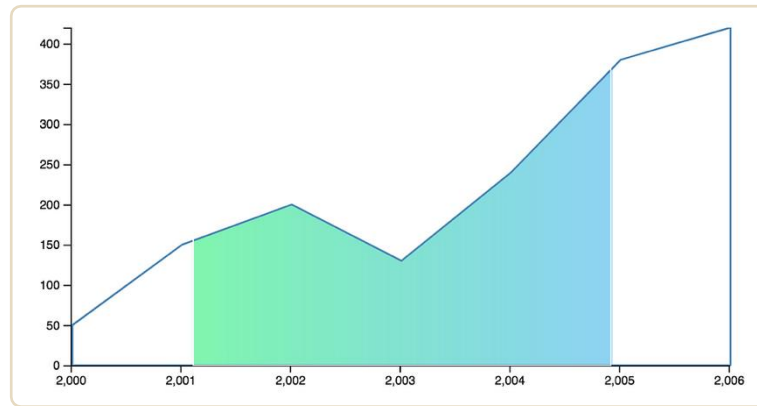
Filtri per selezione

Un **brush** in D3 è uno strumento interattivo che consente di **selezionare un'area specifica** di un grafica, solitamente tracciando il mouse.

Questa selezione può essere utilizzata per **evidenziare, filtrare o ingrandire** una porzione dei dati visualizzati.

D3 mette a disposizione **tre** funzioni:

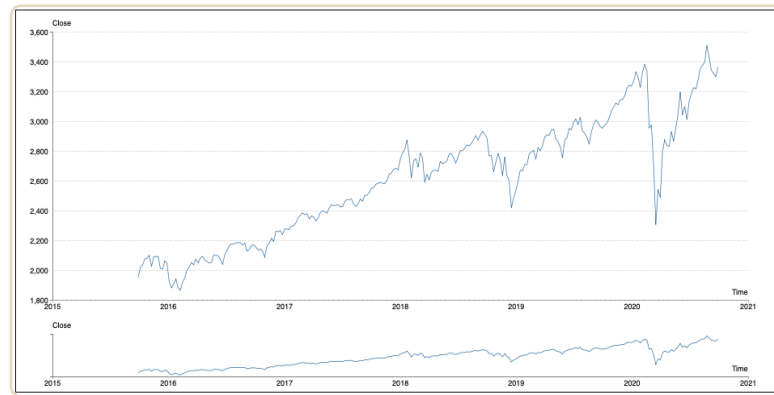
- **D3.brush()**: per selezioni **bidimensionali**.
- **D3.brushX()**: per selezioni lungo l'**asse x**.
- **D3.brushY()**: per selezioni lungo l'**asse y**.



Filtri per selezione

Dati due linechart di dimensioni diverse che rappresentano gli stessi dati, l'obiettivo è:

- Utilizzare il grafico inferiore come **filtro**, perché più piccolo e più difficile da interpretare.
- Evidenziare tramite lo strumento **brush**, nello specifico **brushX**, per evidenziare e visualizzare informazioni specifiche nel grafico superiore.



Definizione della selezione

Il primo passo è **definire l'area** dove il brush può agire. Date le coordinate x, y del linechart inferiore, il pennello può agire dall'angolo in alto a sinistra (l'origine), all'angolo in basso a destra.

Tramite la funzione **d3.brushX()** creiamo il pennello, dandogli come estensione proprio le coordinate appena create.

È necessario definire **cosa succede** sia **durante** che al **termine** della pennellata, nel nostro caso possiamo usare **un'unica funzione**, ma è possibile anche **dividere** i due comportamenti.

brush()

```
// Calcolo le coordinate dell'area del brush
// in modo da posizionarlo correttamente sotto il grafico principale.
const translate = {
  x: config.margin.left + config.origin.x,
  y: config.margin.top + config.origin.y
}

// Le coordinate dell'area del brush sono definite dai punti in alto a sinistra e in basso a destra,
const B = {
  topLeftCorner: [translate.x, translate.y],
  bottomRightCorner: [translate.x + chartWidth, translate.y + chartHeight]
}

// Il brush lavora sull'area del grafico inferiore.
const brush = d3.brushX() brush
  .extent([B.topLeftCorner, B.bottomRightCorner]) function(): any | any {
    // Ogni volta che cambia la selezione del brush,
    // viene chiamata la funzione OnBrushSelectionChange,
    // che converte le coordinate della selezione in un intervallo
    // di tempo e chiama la callback per aggiornare il grafico principale.
    .on("brush end", (e, _) => OnBrushSelectionChange(e, callback, scaleX, translate))

// Aggiungo il brush all'SVG, associandolo al gruppo dedicato e chiamando la funzione brush per attivarlo.
svg
  .append('g')
  .classed('brush-g', true)
  .call(brush)
```

Comportamento del pennello

Tramite **selection** recuperiamo le coordinate **x** di **inizio** selezione e **fine** selezione.

Con la funzione **invert()** otteniamo i valori che rappresentano quelle coordinate nel grafico, nel nostro caso delle date, dato che nell'asse x abbiamo il tempo.

Come visto per lo scatterplot, passiamo questi valori a una funzione **callback** che si occuperà di aggiornare il grafico.

N.B.: il grafico viene ricreato da zero ogni volta che si seleziona qualcosa!

```
function OnBrushSelectionChange(event, callback, scaleX, translate) { Show usages

    // L'area di selezione del brush è definita
    // da due coordinate (x0, x1) che rappresentano
    // i pixel in cui è stata fatta la selezione.

    const area = event.selection;

    // Se non c'è selezione, passo null alla callback
    // per indicare che non c'è filtro da applicare al grafico principale.
    if (!area) {callback(null, null); return}

    // Se c'è una selezione, converto le coordinate pixel
    // in valori temporali usando la scala del grafico del brush.
    const [f_x0, f_x1] = [area[0] - translate.x, area[1] - translate.x]
    const [x0, x1] = [scaleX.invert(f_x0), scaleX.invert(f_x1)]

    // Passo l'intervallo filtrato al grafico principale.
    callback(x0, x1)
}
```

Re-rendering del grafico filtrato

La funzione callback dovrà occuparsi di **filtrare** le informazioni selezionate nel dataset originale e ricreare il grafico con la sola porzione di dati considerata.

Per correttezza è necessario anche gestire quando l'utente clicca sul grafico ma non seleziona **nulla**, in quel caso la funzione callback non fa nulla.

renderLinechartBrush()

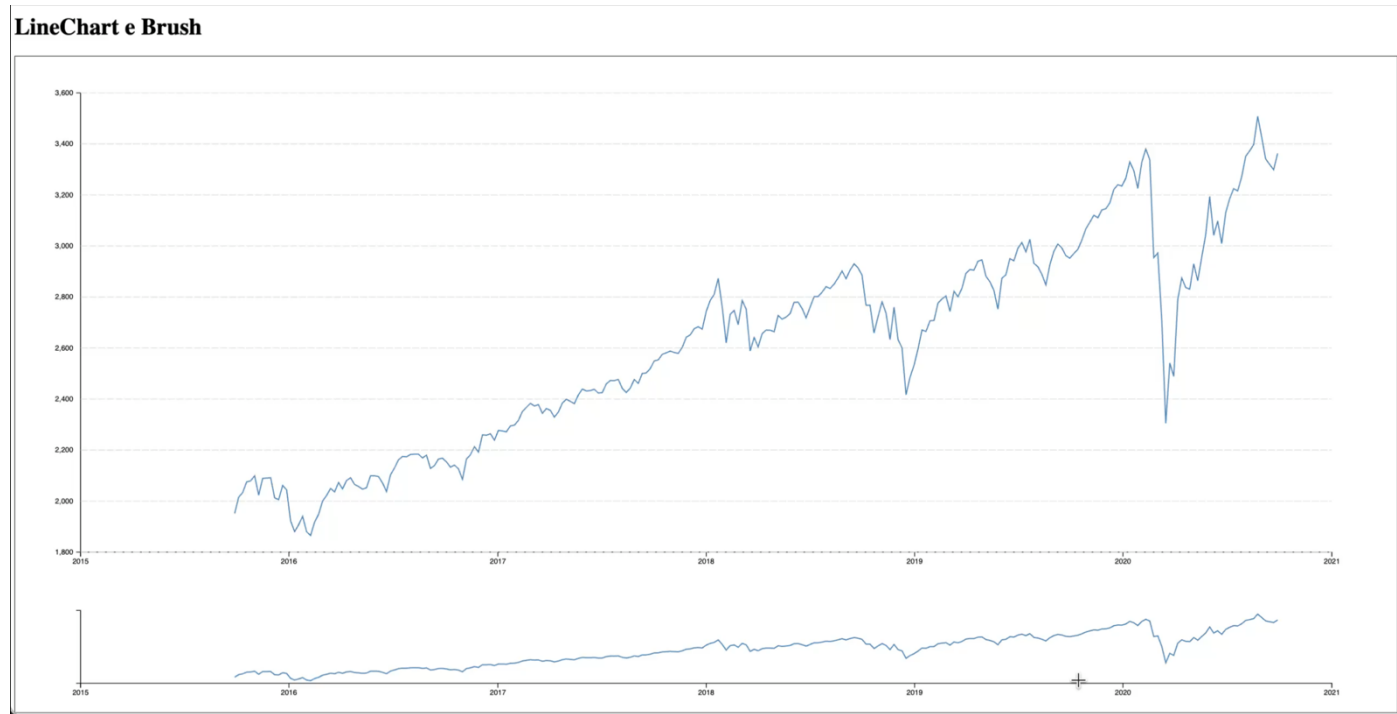
```
linechart(svg, data, config.topChart, ['Time', 'Close'], true, 'main_chart');

// Funzione di callback che viene chiamata ogni volta che cambia la selezione del brush.
function update_chart(x0, x1) { Show usages
  // Se non c'è selezione, non modifico il grafico principale.
  const isClickWithoutSelection = x0 === null && x1 === null
  if (isClickWithoutSelection) {
    return
  }

  // Filtra i dati nel range selezionato e ridisegna il grafico sopra.
  const filtered_data = data.filter(d => d.Time >= x0 && d.Time <= x1)
  // Rimuovo il grafico precedente prima di ridisegnare quello aggiornato.
  d3.select('svg > g#main_chart').remove()
  linechart(svg, filtered_data, config.topChart, ['Time', 'Close'], true, 'main_chart');
}

// Creo il grafico del brush, passando la funzione di callback che aggiorna
// il grafico principale quando cambia la selezione.
brush(svg, data, config.bottomChart, update_chart);
```

Risultato





Esercizi

Esercizio 1:

I punti dello scatterplot possono essere simboli o cerchi in base a un radio button

Esercizio 2:

Quando si fa l'hovering su un punto dello scatterplot, tutti i punti della stessa categoria vengono evidenziati con un bordo nero.

Esercizio 3:

Il main linechart può essere resettato con la distribuzione originale, senza il focus, con un doppio clic o un button.

